Apply Adaboost, XGBoost, and the Gradient Boosting algorithm to the make_classification dataset and compare their performance.

```
from sklearn.datasets import make_classification
```

```
X,y=make_classification(n_samples=1000,n_features=20,random_state=42)
```

```
X.shape
```

```
(1000, 20)
```

```
y.shape
```

```
(1000,)
```

```
from sklearn.model_selection import train_test_split
X_train,X_test,y_train,y_test=train_test_split(X,y,test_size=0.2,random_state=42)
```

```
print(X_train.shape)
print(X_test.shape)
```

```
(800, 20)
(200, 20)
```

```
from sklearn.ensemble import AdaBoostClassifier,GradientBoostingClassifier
```

AdaBoost Classifier

```
ada=AdaBoostClassifier(n_estimators=10,random_state=42)
ada.fit(X_train,y_train)
```

```
AdaBoostClassifier
AdaBoostClassifier(n_estimators=10, random_state=42)
```

```
y_pred_ada = ada.predict(X_test)
```

Gradient Boosting Classifier

```
gbc=GradientBoostingClassifier(n_estimators=10,learning_rate=0.1,random_state=42)
gbc.fit(X_train,y_train)
```

```
GradientBoostingClassifier
GradientBoostingClassifier(n_estimators=10, random_state=42)
```

```
y_pred_gb=gbc.predict(X_test)
```

XGBoost classifier

```
from xgboost import XGBClassifier
xgb=XGBClassifier(n_estimators=10)
xgb.fit(X_train,y_train)
```

```
XGBClassifier
XGBClassifier(base_score=None, booster=None, callbacks=None,
              colsample_bylevel=None, colsample_bynode=None,
              colsample_bytree=None, device=None, early_stopping_rounds=None,
              enable_categorical=False, eval_metric=None, feature_types=None,
              feature_weights=None, gamma=None, grow_policy=None,
              importance_type=None, interaction_constraints=None,
              learning_rate=None, max_bin=None, max_cat_threshold=None,
              max_cat_to_onehot=None, max_delta_step=None, max_depth=None,
              max_leaves=None, min_child_weight=None, missing=nan,
              monotone_constraints=None, multi_strategy=None, n_estimators=10,
              n_jobs=None, num_parallel_tree=None, ...)
```

```
y_pred_xgb=xgb.predict(X_test)
```

```
from sklearn.metrics import accuracy_score

print("AdaBoost Accuracy:",accuracy_score(y_test,y_pred_ada))
print("Gradient Boosting Accuracy:",accuracy_score(y_test,y_pred_gb))
print("XGBoost Accuracy:",accuracy_score(y_test,y_pred_xgb))
```

```
AdaBoost Accuracy: 0.86
Gradient Boosting Accuracy: 0.88
XGBoost Accuracy: 0.895
```

```
from sklearn.metrics import classification_report

print("AdaBoost Classifier: \n",classification_report(y_test,y_pred_ada))
print("Gradient Boosting Classifier:\n",classification_report(y_test,y_pred_gb))
print("XGBoost Classifier:\n",classification_report(y_test,y_pred_xgb))
```

```
AdaBoost Classifier:
               precision    recall  f1-score   support

           0       0.85      0.85      0.85        93
           1       0.87      0.87      0.87       107

    accuracy                           0.86       200
   macro avg       0.86      0.86      0.86       200
weighted avg       0.86      0.86      0.86       200

Gradient Boosting Classifier:
               precision    recall  f1-score   support

           0       0.83      0.94      0.88        93
           1       0.94      0.83      0.88       107

    accuracy                           0.88       200
   macro avg       0.88      0.88      0.88       200
weighted avg       0.89      0.88      0.88       200

XGBoost Classifier:
               precision    recall  f1-score   support

           0       0.85      0.95      0.89        93
           1       0.95      0.85      0.90       107

    accuracy                           0.90       200
   macro avg       0.90      0.90      0.89       200
weighted avg       0.90      0.90      0.90       200
```